

# Package ‘ndm’

March 15, 2026

**Title** Neural Disease Modeling Software

**Version** 0.0.1

**Author** Connor T. Jerzak [aut, cre]

**Maintainer** Connor T. Jerzak <connor.jerzak@austin.utexas.edu>

**Description** Provides an R-first interface for neural disease modeling with a reticulate-managed JAX backend. The package includes backend bootstrap helpers, built-in epidemiological model specifications, TFRecord-consuming data interfaces, a bundled self-contained analysis runtime, and package-native orchestration helpers for model execution.

**Depends** R (>= 4.1.0)

**Imports** data.table, ndmdatasets, rlang, reticulate, utils

**Suggests** fastmatch, progress, rapply, testthat (>= 3.0.0), yaml, zip,  
zoo

**License** GPL-3

**Encoding** UTF-8

**Config/testthat/edition** 3

**RoxygenNote** 7.3.3

**NeedsCompilation** no

## Contents

|                                      |    |
|--------------------------------------|----|
| ndm-package . . . . .                | 2  |
| ndm_build_model . . . . .            | 3  |
| ndm_check_backend . . . . .          | 4  |
| ndm_create_config . . . . .          | 6  |
| ndm_create_real_run_config . . . . . | 7  |
| ndm_create_tfrecord_parser . . . . . | 9  |
| ndm_model_spec_presets . . . . .     | 11 |
| ndm_new_runtime_env . . . . .        | 12 |
| ndm_predict . . . . .                | 13 |
| ndm_prepare_runtime . . . . .        | 14 |
| ndm_print . . . . .                  | 15 |
| ndm_tfrecord_fields_real . . . . .   | 16 |
| ndm_tf_batch_to_r . . . . .          | 16 |

|              |           |
|--------------|-----------|
| <b>Index</b> | <b>19</b> |
|--------------|-----------|

## Description

ndm is an R-first package for neural disease modeling with a reticulate-managed JAX backend. The package currently provides backend setup, epidemic model spec import/export, TFRecord readers, a bundled internal analysis runtime, and package-native runtime/model helpers.

## Details

The current implementation preserves both historical model families, "DecoderOnly" and "NeuralODE", while defaulting to "DecoderOnly". The supported backbone is the transformer path only. Latent attention and Mamba are intentionally excluded.

The package treats the structured `ndm_model_spec` object as the canonical internal representation and supports `.tex` import/export for compatibility with the original epidemic specification workflow.

## Key functions

- `ndm_create_config`
- `ndm_build_backend`
- `ndm_initialize_backend`
- `ndm_model_spec`
- `ndm_model_spec_from_tex`
- `ndm_model_spec_to_tex`
- `ndm_read_tfrecord_dataset`
- `ndm_load_tfrecord_bundle`
- `ndm_prepare_runtime`
- `ndm_prepare_data`
- `ndm_build_model`
- `ndm_train`
- `ndm_predict`
- `ndm_loss`
- `ndm_fit`

## Author(s)

Connor T. Jerzak <connor.jerzak@austin.utexas.edu>

**Description**

These wrappers layer a small R API over local Phase 1 model build and training scripts.

**Usage**

```
ndm_build_model(
  runtime_env,
  model_type = c("DecoderOnly", "NeuralODE"),
  model_spec = NULL,
  backbone = "transformer",
  runtime_globals = list()
)

## S3 method for class 'ndm_model'
print(x, ...)

ndm_train(x, run_define = TRUE, run_loop = TRUE)

## S3 method for class 'ndm_trained_model'
print(x, ...)

ndm_fit(
  config = ndm_create_config(),
  model_spec = NULL,
  data_generator = c("sim", "real", "multidisease"),
  runtime_globals = list(),
  data_globals = list(),
  build_globals = list(),
  train_globals = list(),
  run_define = TRUE,
  run_loop = TRUE
)
```

**Arguments**

|                 |                                                                                                                                                                                                                          |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| runtime_env     | Runtime environment containing the sourced legacy helper code and data globals.                                                                                                                                          |
| model_type      | Model family to build. Either <code>"DecoderOnly"</code> or <code>"NeuralODE"</code> .                                                                                                                                   |
| model_spec      | Optional <code>'ndm_model_spec'</code> object used to override the model TeX specification supplied to the runtime model builder.                                                                                        |
| backbone        | Backbone family. Phase 1 supports <code>"transformer"</code> only.                                                                                                                                                       |
| runtime_globals | Named list of additional globals assigned during the relevant runtime stage. <code>'ndm_build_model()'</code> uses them before building the model, while <code>'ndm_fit()'</code> uses them before sourcing the runtime. |

|                             |                                                                                                                                                       |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>x</code>              | Either an <code>'ndm_model'</code> , an <code>'ndm_trained_model'</code> , or a prepared runtime environment, depending on the function being called. |
| <code>...</code>            | Unused; included for S3 method compatibility.                                                                                                         |
| <code>run_define</code>     | Logical scalar indicating whether the training definition script should be sourced.                                                                   |
| <code>run_loop</code>       | Logical scalar indicating whether the training loop script should be sourced.                                                                         |
| <code>config</code>         | An object of class <code>'ndm_config'</code> , usually created by <code>'ndm_create_config()'</code> .                                                |
| <code>data_generator</code> | Which local data generator to source before calling <code>'ndm_build_model()'</code> inside <code>'ndm_fit()'</code> .                                |
| <code>data_globals</code>   | Named list of globals assigned before sourcing the data generator in <code>'ndm_fit()'</code> .                                                       |
| <code>build_globals</code>  | Named list of globals assigned before building the model in <code>'ndm_fit()'</code> .                                                                |
| <code>train_globals</code>  | Named list of globals assigned immediately before training in <code>'ndm_fit()'</code> .                                                              |

### Value

`'ndm_build_model()'` returns an object of class `'ndm_model'`. `'ndm_train()'` and `'ndm_fit()'` return an object of class `'ndm_trained_model'`.

`'print.ndm_model()'` prints a brief summary of an `'ndm_model'` object and invisibly returns `'x'`.

`'print.ndm_trained_model()'` prints a brief summary of an `'ndm_trained_model'` object and invisibly returns `'x'`.

### Examples

```
cfg <- ndm_create_config()
spec <- ndm_model_spec()
## Not run:
runtime_env <- ndm_prepare_runtime(cfg)
ndm_prepare_data(runtime_env, generator = "sim")
model <- ndm_build_model(runtime_env, model_spec = spec)
trained <- ndm_train(model)

## End(Not run)
```

---

ndm\_check\_backend

---

*Provision and initialize the Python backend*


---

### Description

These helpers manage the reticulate-backed Python environment used by ndm. Use `'ndm_build_backend()'` to provision dependencies, `'ndm_check_backend()'` to verify availability, and `'ndm_initialize_backend()'` to import the active JAX stack into R.

### Usage

```
ndm_check_backend(
  conda_env = "ndm_software_env",
  conda = "auto",
  modules = c("jax", "numpy", "optax", "equinox", "diffraction")
)
```

```

ndm_build_backend(
    conda_env = "ndm_software_env",
    conda = "auto",
    python_version = "3.13",
    include_tensorflow = TRUE,
    extra_packages = c("optax", "equinox", "diffrax")
)

ndm_initialize_backend(
    conda_env = "ndm_software_env",
    conda = "auto",
    conda_env_required = TRUE,
    float_type = c("32", "64"),
    import_tensorflow = TRUE
)

ndm_backend_modules()

```

### Arguments

|                                 |                                                                                                                   |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>conda_env</code>          | Name of the conda environment that should contain the JAX runtime.                                                |
| <code>conda</code>              | Conda executable to use. <code>"auto"</code> delegates discovery to reticulate.                                   |
| <code>modules</code>            | Character vector of Python module names that must be importable for the backend to be considered healthy.         |
| <code>python_version</code>     | Python version requested when creating the conda environment.                                                     |
| <code>include_tensorflow</code> | Logical scalar indicating whether TensorFlow should be installed alongside JAX.                                   |
| <code>extra_packages</code>     | Additional Python packages to install with <code>'pip'</code> .                                                   |
| <code>conda_env_required</code> | Passed through to <code>'reticulate::use_condaenv()'</code> as the <code>'required'</code> flag.                  |
| <code>float_type</code>         | Floating point precision used when configuring JAX. Either <code>"32"</code> or <code>"64"</code> .               |
| <code>import_tensorflow</code>  | Logical scalar indicating whether TensorFlow should be imported when it is available in the selected environment. |

### Value

`'ndm_check_backend()'` returns `'TRUE'` invisibly when the requested environment and modules are available. It returns `'NULL'` after printing a diagnostic message when the backend is not ready.

`'ndm_build_backend()'` invisibly returns the conda environment name after attempting to provision the requested packages.

`'ndm_initialize_backend()'` returns an object of class `'ndm_backend'`. The returned list contains imported Python modules, the configured float type, and a small compatibility shim used by the legacy runtime.

`'ndm_backend_modules()'` returns the currently initialized `'ndm_backend'` object.

## Examples

```
ndm_check_backend()

## Not run:
ndm_build_backend()

## End(Not run)

## Not run:
backend <- ndm_initialize_backend()
backend$default_backend

## End(Not run)

## Not run:
ndm_initialize_backend()
ndm_backend_modules()

## End(Not run)
```

---

|                   |                                             |
|-------------------|---------------------------------------------|
| ndm_create_config | <i>Create runtime configuration objects</i> |
|-------------------|---------------------------------------------|

---

## Description

Configuration objects collect the runtime options that are threaded through the higher-level orchestration helpers such as ‘ndm\_prepare\_runtime()’ and ‘ndm\_fit()’.

## Usage

```
ndm_create_config(
  model_type = c("DecoderOnly", "NeuralODE"),
  backbone = "transformer",
  float_type = c("32", "64"),
  force_to_gpu = TRUE,
  resave_tfrecords = FALSE,
  gpu_mem_frac = NULL,
  ...
)

## S3 method for class 'ndm_config'
print(x, ...)
```

## Arguments

|              |                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------|
| model_type   | Model family metadata. Use “DecoderOnly” or “NeuralODE”.                                        |
| backbone     | Backbone family. Phase 1 supports “transformer” only.                                           |
| float_type   | Floating point precision used when initializing the backend. Use “32” or “64”.                  |
| force_to_gpu | Logical scalar indicating whether the runtime should try to place arrays on GPU when available. |

|                    |                                                                         |
|--------------------|-------------------------------------------------------------------------|
| resave_tfrerecords | Logical scalar preserved for compatibility with the legacy runtime.     |
| gpu_mem_frac       | Optional GPU memory fraction forwarded into the runtime bootstrap code. |
| ...                | Unused; included for S3 method compatibility.                           |
| x                  | An 'ndm_config' object.                                                 |

### Value

'ndm\_create\_config()' returns an object of class 'ndm\_config'.

'print.ndm\_config()' prints a brief summary of an 'ndm\_config' object and invisibly returns 'x'.

### Examples

```
cfg <- ndm_create_config()
print(cfg)
```

---

ndm\_create\_real\_run\_config

*Package-native run configurations and workflow runners*

---

### Description

Package-native configuration constructors and workflow runners for the self-contained real, simulation, and multidisease execution paths.

### Usage

```
ndm_create_real_run_config(
  project_root = getwd(),
  analysis_name = "RealApril15",
  grid = NULL,
  grid_file = file.path("Data", "RunGrids", "RealGrids", sprintf("RealGrid_%s.csv",
    analysis_name)),
  outer = 3L,
  model_type = NULL,
  respect_grid_model_type = FALSE,
  resave_tfrerecords = FALSE,
  tfrecord_dir = file.path("Data", "RunTFRecords", "RealTFRecords",
    analysis_name),
  raw_data_dir = file.path("Data", "MainData"),
  outcome_metric = "inc_death",
  data_subset = "high_income",
  dry_run = FALSE
)

ndm_create_sim_run_config(
  project_root = getwd(),
  analysis_name = "BigSimsLatest",
  grid = NULL,
```

```

grid_file = file.path("Data", "RunGrids", "SimGrids", sprintf("SimGrid_%s.csv",
  analysis_name)),
outer = 1L,
model_type = NULL,
respect_grid_model_type = FALSE,
resave_tfrerecords = TRUE,
tfrecord_dir = file.path("Data", "RunTFRecords", "SimTFRecords",
  analysis_name),
dry_run = FALSE
)

ndm_create_multidisease_run_config(
  project_root = getwd(),
  analysis_name = "RealLatest",
  grid = NULL,
  grid_file = file.path("Data", "RunGrids", "RealGrids", sprintf("RealGrid_%s.csv",
    analysis_name)),
  outer = 1L,
  model_type = NULL,
  respect_grid_model_type = FALSE,
  resave_tfrerecords = FALSE,
  tfrecord_dir = file.path("Data", "RunTFRecords", "RealTFRecords",
    analysis_name),
  outcome_metric = "CountValue",
  data_subset = "all",
  disease_names = c("Covid", "Flu"),
  data_format = "IHME",
  dry_run = FALSE
)

ndm_run_real(config = ndm_create_real_run_config())

ndm_run_sim(config = ndm_create_sim_run_config())

ndm_run_multidisease(config = ndm_create_multidisease_run_config())

```

### Arguments

|                                      |                                                                                                       |
|--------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>project_root</code>            | Working project directory used for grids, TFRecords, and outputs.                                     |
| <code>analysis_name</code>           | Analysis label threaded through output paths.                                                         |
| <code>grid</code>                    | Optional in-memory grid data frame. When supplied, <code>grid_file</code> must be <code>NULL</code> . |
| <code>grid_file</code>               | Optional CSV path for the run grid.                                                                   |
| <code>outer</code>                   | Integer vector of outer-iteration rows to execute.                                                    |
| <code>model_type</code>              | Optional model family override.                                                                       |
| <code>respect_grid_model_type</code> | Logical scalar indicating whether grid rows may override the requested model type.                    |
| <code>resave_tfrerecords</code>      | Logical scalar controlling TFRecord regeneration.                                                     |
| <code>tfrecord_dir</code>            | Optional TFRecord output directory.                                                                   |

|                |                                                                                              |
|----------------|----------------------------------------------------------------------------------------------|
| raw_data_dir   | Real-data input directory.                                                                   |
| outcome_metric | Outcome metric name for real-data or multidisease runs.                                      |
| data_subset    | Optional real-data subset name.                                                              |
| disease_names  | Character vector of multidisease names.                                                      |
| data_format    | Multidisease data format.                                                                    |
| dry_run        | Logical scalar indicating whether the run should stop after resolving paths and grid rows.   |
| config         | A package-native run configuration created by the matching ndm_create_*_run_config() helper. |

### Value

The ndm\_create\_\*\_run\_config() helpers return classed lists describing the requested workflow. The ndm\_run\_\*() functions return the underlying workflow result, or a dry-run preview when config\$dry\_run is TRUE.

### Examples

```
## Not run:
cfg <- ndm_create_sim_run_config(
  project_root = getwd(),
  grid_file = "Data/RunGrids/SimGrids/SimGrid_BigSimsLatest.csv",
  outer = 1L,
  dry_run = TRUE
)

ndm_run_sim(cfg)

## End(Not run)
```

---

ndm\_create\_tfrecord\_parser

*Create and read TensorFlow TFRecord datasets*

---

### Description

These wrappers construct parsers for the ndm TFRecord schema and use them to build TensorFlow datasets or eagerly collect batches into R.

### Usage

```
ndm_create_tfrecord_parser(field_names, dtype_map = NULL, tensorflow = NULL)

ndm_read_tfrecord_dataset(
  file,
  batch_size,
  field_names,
  dtype_map = NULL,
  max_examples = NULL,
  shuffle = FALSE,
```

```

    buffer_scaler = 10L,
    tensorflow = NULL
  )

  ndm_collect_tfrecord_batches(
    file,
    batch_size,
    field_names,
    dtype_map = NULL,
    max_batches = NULL,
    max_examples = NULL,
    shuffle = FALSE,
    buffer_scaler = 10L,
    tensorflow = NULL
  )

```

### Arguments

|               |                                                                                                                                                                                                |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| field_names   | Character vector of TFRecord field names that should be parsed from each example.                                                                                                              |
| dtype_map     | Optional named vector or list mapping 'field_names' to TensorFlow dtype names or TensorFlow dtype objects.                                                                                     |
| tensorflow    | Optional TensorFlow module. When omitted, ndm uses the active backend's TensorFlow module when available and otherwise imports TensorFlow from the active reticulate environment at call time. |
| file          | Path to the '.tfrecord' file to read.                                                                                                                                                          |
| batch_size    | Batch size used when constructing the TensorFlow dataset.                                                                                                                                      |
| max_examples  | Optional maximum number of examples to consume from the file before batching.                                                                                                                  |
| shuffle       | Logical scalar indicating whether the dataset should be shuffled.                                                                                                                              |
| buffer_scaler | Integer multiplier used to derive the TensorFlow shuffle buffer size from 'batch_size'.                                                                                                        |
| max_batches   | Optional maximum number of batches to collect when using 'ndm_collect_tfrecord_batches()'.                                                                                                     |

### Value

'ndm\_create\_tfrecord\_parser()' returns a parser function compatible with 'tf\$data\$Dataset\$map()'.  
 'ndm\_read\_tfrecord\_dataset()' returns a TensorFlow dataset object. 'ndm\_collect\_tfrecord\_batches()' returns a list of parsed batches.

### Examples

```

fields <- ndm_tfrecord_fields_real()
parser <- ndm_create_tfrecord_parser(fields, ndm_tfrecord_dtype_map("real"))
is.function(parser)

```

---

ndm\_model\_spec\_presets

*Inspect and create epidemic model specifications*


---

## Description

The package ships a small registry of built-in compartmental model specifications and also supports importing custom ‘.tex’ model definitions from disk. The structured ‘ndm\_model\_spec’ object is the canonical representation used by the runtime wrappers.

## Usage

```
ndm_model_spec_presets()

ndm_model_spec(
  preset = NULL,
  compartments = c("seir", "seirs"),
  dynamic_beta = NULL,
  dynamic_global = FALSE,
  multi_outcome = FALSE,
  model_type = c("DecoderOnly", "NeuralODE"),
  time_varying_terms = NULL,
  endogenous_terms = NULL,
  tex_path = NULL
)

ndm_model_spec_from_tex(path, model_type = c("DecoderOnly", "NeuralODE"))

ndm_model_spec_to_tex(spec, path = NULL)

## S3 method for class 'ndm_model_spec'
print(x, ...)
```

## Arguments

|                    |                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| preset             | Optional preset name from ‘ndm_model_spec_presets()’. When omitted, the remaining arguments are used to infer a built-in preset. |
| compartments       | Compartment family used when inferring a built-in preset.                                                                        |
| dynamic_beta       | Logical scalar controlling whether the local transmission rate is time-varying for built-in presets.                             |
| dynamic_global     | Logical scalar controlling whether the global rates are time-varying for built-in presets.                                       |
| multi_outcome      | Logical scalar indicating whether the observation model should include multiple outcomes.                                        |
| model_type         | Model family metadata attached to the returned specification. Either “DecoderOnly” or “NeuralODE”.                               |
| time_varying_terms | Optional character vector overriding the default time-varying term metadata stored on the returned spec.                         |

|                  |                                                                                                                    |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| endogenous_terms | Optional character vector overriding the default endogenous term metadata stored on the returned spec.             |
| tex_path         | Optional path to a custom '.tex' file. When supplied, 'ndm_model_spec()' delegates to 'ndm_model_spec_from_tex()'. |
| path             | Path to a '.tex' model specification file on disk.                                                                 |
| spec             | An 'ndm_model_spec' object.                                                                                        |
| x                | An 'ndm_model_spec' object.                                                                                        |
| ...              | Unused; included for S3 method compatibility.                                                                      |

### Value

'ndm\_model\_spec\_presets()' returns a data frame describing the built-in presets bundled with the package.

'ndm\_model\_spec()' returns an object of class 'ndm\_model\_spec'. Built-in presets include metadata such as the packaged source path and the normalized TeX contents.

'ndm\_model\_spec\_from\_tex()' returns an 'ndm\_model\_spec' object whose 'source\_path' points at the supplied file. When the basename matches a built-in preset, the preset metadata is reused.

'ndm\_model\_spec\_to\_tex()' returns the TeX text as a single character string when 'path' is 'NULL'. When 'path' is supplied, it writes the specification to disk and invisibly returns the normalized output path.

'print.ndm\_model\_spec()' prints a brief summary of an 'ndm\_model\_spec' object and invisibly returns 'x'.

### Examples

```
ndm_model_spec_presets()

spec <- ndm_model_spec()
print(spec)

spec <- ndm_model_spec()
tex_path <- tempfile(fileext = ".tex")
ndm_model_spec_to_tex(spec, tex_path)
ndm_model_spec_from_tex(tex_path)

spec <- ndm_model_spec()
tex <- ndm_model_spec_to_tex(spec)
nchar(tex) > 0
```

---

ndm\_new\_runtime\_env     *Create and populate runtime environments*

---

### Description

These helpers manage isolated environments used to source local model runtime code and seed it with R values.

**Usage**

```
ndm_new_runtime_env(parent = globalenv())

ndm_set_runtime_globals(env, values, overwrite = TRUE)
```

**Arguments**

|           |                                                                                                                                                                                     |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| parent    | Parent environment for a new runtime environment. Defaults to ‘globalenv()’ so bundled runtime scripts can resolve attached-package functions the same way the legacy workflow did. |
| env       | Runtime environment that should receive new bindings.                                                                                                                               |
| values    | Named list of values to assign into ‘env’.                                                                                                                                          |
| overwrite | Logical scalar indicating whether existing bindings in ‘env’ should be overwritten.                                                                                                 |

**Value**

‘ndm\_new\_runtime\_env()’ returns an environment of class ‘ndm\_runtime\_env’. ‘ndm\_set\_runtime\_globals()’ invisibly returns ‘env’.

**Examples**

```
env <- ndm_new_runtime_env()
ndm_set_runtime_globals(env, list(example_value = 1L))
env$example_value
```

---

ndm\_predict

---

*Generate predictions or evaluate the training loss*


---

**Description**

These helpers call the runtime prediction and loss functions stored in a prepared runtime environment.

**Usage**

```
ndm_predict(x, batch = NULL, inference = TRUE, seed = 1L, update_state = TRUE)

ndm_loss(
  x,
  batch = NULL,
  y = NULL,
  y_mask = NULL,
  iteration = 1L,
  seed = 1L,
  update_state = TRUE
)
```

**Arguments**

|              |                                                                                                                                                                                                            |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| x            | Either an 'ndm_model', an 'ndm_trained_model', or a prepared runtime environment that already contains the required runtime objects.                                                                       |
| batch        | Optional batch object. Supply either a named TFRecord-style list or the four-element packaged list returned by 'ndm_batch_to_model_inputs()'. When 'NULL', the runtime must already contain 'batch_l_cal'. |
| inference    | Logical scalar indicating whether 'ndm_predict()' should use the inference prediction path or the training prediction path.                                                                                |
| seed         | Integer seed used to derive the per-example JAX RNG keys.                                                                                                                                                  |
| update_state | Logical scalar indicating whether the runtime 'state' object should be updated with the returned state.                                                                                                    |
| y            | Optional observed outcome tensor passed to 'ndm_loss()'. When omitted, 'YTrue_out' is inferred from 'batch'.                                                                                               |
| y_mask       | Optional outcome mask tensor passed to 'ndm_loss()'. When omitted, 'YTrue_out_mask' is inferred from 'batch'.                                                                                              |
| iteration    | Training iteration number forwarded to the runtime loss function.                                                                                                                                          |

**Value**

'ndm\_predict()' returns the prediction object produced by the runtime. 'ndm\_loss()' returns the loss object produced by the runtime.

**Examples**

```
## Not run:
preds <- ndm_predict(model, batch = batch_l_cal)
loss <- ndm_loss(model, batch = batch_l_cal)

## End(Not run)
```

---

ndm\_prepare\_runtime      *Prepare a local runtime for execution*

---

**Description**

These wrappers load caller-supplied runtime scripts into an isolated environment and then source the selected data generator.

**Usage**

```
ndm_prepare_runtime(
  config = ndm_create_config(),
  runtime_env = ndm_new_runtime_env(),
  runtime_globals = list()
)

ndm_prepare_data(
  runtime_env,
  generator = c("sim", "real", "multidisease"),
  runtime_globals = list()
)
```

**Arguments**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| config          | An object of class 'ndm_config', usually created by 'ndm_create_config()'.      |
| runtime_env     | Runtime environment that should receive the sourced objects.                    |
| runtime_globals | Named list of additional bindings to assign before the runtime code is sourced. |
| generator       | Which data generator to source into 'runtime_env'.                              |

**Value**

'ndm\_prepare\_runtime()' invisibly returns 'runtime\_env' after loading the package-owned helper and backend code.

'ndm\_prepare\_data()' invisibly returns 'runtime\_env' after sourcing the requested data generator.

**Examples**

```
env <- ndm_new_runtime_env()
cfg <- ndm_create_config()
## Not run:
ndm_prepare_runtime(cfg, env)

## End(Not run)

env <- ndm_new_runtime_env()
ndm_set_runtime_globals(env, list(example_value = 1L))
```

---

ndm\_print

---

*Print a timestamped diagnostic message*


---

**Description**

'ndm\_print()' is a small logging helper used throughout the package-facing orchestration code.

**Usage**

```
ndm_print(text, quiet = FALSE)
```

**Arguments**

|       |                                                                |
|-------|----------------------------------------------------------------|
| text  | Text to print.                                                 |
| quiet | Logical scalar indicating whether output should be suppressed. |

**Value**

The input 'text', invisibly.

**Examples**

```
ndm_print("starting")
```

---

ndm\_tfrecord\_fields\_real

*Inspect TFRecord field contracts*


---

### Description

These helpers describe the field names and default TensorFlow dtypes expected by the packaged TFRecord readers.

### Usage

```
ndm_tfrecord_fields_real()
```

```
ndm_tfrecord_fields_sim()
```

```
ndm_tfrecord_dtype_map(kind = c("real", "sim"))
```

### Arguments

|      |                                                                                                    |
|------|----------------------------------------------------------------------------------------------------|
| kind | TFRecord schema to inspect. Use "real" for observed-data records and "sim" for simulation records. |
|------|----------------------------------------------------------------------------------------------------|

### Value

'ndm\_tfrecord\_fields\_real()' and 'ndm\_tfrecord\_fields\_sim()' return character vectors of field names. 'ndm\_tfrecord\_dtype\_map()' returns a named character vector mapping each field to its default TensorFlow dtype.

### Examples

```
ndm_tfrecord_fields_real()
ndm_tfrecord_dtype_map("sim")
```

---

ndm\_tf\_batch\_to\_r

*Convert and bundle TFRecord batches*


---

### Description

These helpers convert TensorFlow batches to native R or JAX objects, package them into the shape expected by the legacy runtime, and attach dataset handles to a runtime environment.

### Usage

```
ndm_tf_batch_to_r(batch)
```

```
ndm_tf_batch_to_jax(batch, backend = NULL)
```

```
ndm_batch_to_model_inputs(batch)
```

```

ndm_load_tfrecord_bundle(
    train_file,
    inference_file = NULL,
    batch_size,
    kind = c("real", "sim"),
    dtype_map = NULL,
    tensorflow = NULL,
    max_train_examples = NULL,
    max_inference_examples = NULL,
    shuffle_train = TRUE,
    shuffle_inference = FALSE
)

ndm_attach_tfrecord_bundle(
    env,
    bundle,
    backend = NULL,
    calibration_source = c("inference", "train")
)

```

## Arguments

|                        |                                                                                                                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| batch                  | A parsed TFRecord batch or a nested list containing TensorFlow tensors.                                                                                                                        |
| backend                | Optional 'ndm_backend' object. When omitted, 'ndm_tf_batch_to_jax()' and 'ndm_attach_tfrecord_bundle()' use the active backend from 'ndm_backend_modules()' when available.                    |
| train_file             | Path to the training TFRecord file.                                                                                                                                                            |
| inference_file         | Optional path to an inference TFRecord file.                                                                                                                                                   |
| batch_size             | Batch size used when constructing the training and inference datasets.                                                                                                                         |
| kind                   | TFRecord schema to use when reading 'train_file' and 'inference_file'.                                                                                                                         |
| dtype_map              | Optional named dtype mapping passed through to the TFRecord readers.                                                                                                                           |
| tensorflow             | Optional TensorFlow module. When omitted, ndm uses the active backend's TensorFlow module when available and otherwise imports TensorFlow from the active reticulate environment at call time. |
| max_train_examples     | Optional cap on the number of training examples to read.                                                                                                                                       |
| max_inference_examples | Optional cap on the number of inference examples to read.                                                                                                                                      |
| shuffle_train          | Logical scalar indicating whether the training dataset should be shuffled.                                                                                                                     |
| shuffle_inference      | Logical scalar indicating whether the inference dataset should be shuffled.                                                                                                                    |
| env                    | Runtime environment that should receive dataset handles and the preview calibration batch.                                                                                                     |
| bundle                 | An object of class 'ndm_tfrecord_bundle' returned by 'ndm_load_tfrecord_bundle()'.                                                                                                             |
| calibration_source     | Which dataset should supply the preview batch used to seed runtime globals such as 'batch_l_cal'.                                                                                              |

**Value**

'ndm\_tf\_batch\_to\_r()' recursively converts a TensorFlow batch to R arrays and lists. 'ndm\_tf\_batch\_to\_jax()' recursively converts tensors into JAX arrays. 'ndm\_batch\_to\_model\_inputs()' returns the packaged four-element list expected by the legacy model code. 'ndm\_load\_tfrecord\_bundle()' returns an 'ndm\_tfrecord\_bundle' object. 'ndm\_attach\_tfrecord\_bundle()' invisibly returns 'env'.

**Examples**

```
batch <- list(  
  XPred = array(0, c(2, 3, 4)),  
  XPred_mask = array(TRUE, c(2, 3, 4)),  
  Context = array(0, c(2, 1, 4)),  
  Context_mask = array(TRUE, c(2, 1, 4)),  
  location_id_numeric = matrix(1L, nrow = 2),  
  time_id_numeric = matrix(1L, nrow = 2)  
)  
packaged <- ndm_batch_to_model_inputs(batch)  
length(packaged)
```

# Index

ndm (ndm-package), 2  
ndm-package, 2  
ndm\_attach\_tfrecord\_bundle  
    (ndm\_tf\_batch\_to\_r), 16  
ndm\_backend\_modules  
    (ndm\_check\_backend), 4  
ndm\_batch\_to\_model\_inputs  
    (ndm\_tf\_batch\_to\_r), 16  
ndm\_build\_backend, 2  
ndm\_build\_backend (ndm\_check\_backend), 4  
ndm\_build\_model, 2, 3  
ndm\_check\_backend, 4  
ndm\_collect\_tfrecord\_batches  
    (ndm\_create\_tfrecord\_parser), 9  
ndm\_create\_config, 2, 6  
ndm\_create\_multidisease\_run\_config  
    (ndm\_create\_real\_run\_config), 7  
ndm\_create\_real\_run\_config, 7  
ndm\_create\_sim\_run\_config  
    (ndm\_create\_real\_run\_config), 7  
ndm\_create\_tfrecord\_parser, 9  
ndm\_fit, 2  
ndm\_fit (ndm\_build\_model), 3  
ndm\_initialize\_backend, 2  
ndm\_initialize\_backend  
    (ndm\_check\_backend), 4  
ndm\_load\_tfrecord\_bundle, 2  
ndm\_load\_tfrecord\_bundle  
    (ndm\_tf\_batch\_to\_r), 16  
ndm\_loss, 2  
ndm\_loss (ndm\_predict), 13  
ndm\_model\_spec, 2  
ndm\_model\_spec  
    (ndm\_model\_spec\_presets), 11  
ndm\_model\_spec\_from\_tex, 2  
ndm\_model\_spec\_from\_tex  
    (ndm\_model\_spec\_presets), 11  
ndm\_model\_spec\_presets, 11  
ndm\_model\_spec\_to\_tex, 2  
ndm\_model\_spec\_to\_tex  
    (ndm\_model\_spec\_presets), 11  
ndm\_new\_runtime\_env, 12  
ndm\_predict, 2, 13  
ndm\_prepare\_data, 2  
ndm\_prepare\_data (ndm\_prepare\_runtime),  
    14  
ndm\_prepare\_runtime, 2, 14  
ndm\_print, 15  
ndm\_read\_tfrecord\_dataset, 2  
ndm\_read\_tfrecord\_dataset  
    (ndm\_create\_tfrecord\_parser), 9  
ndm\_run\_multidisease  
    (ndm\_create\_real\_run\_config), 7  
ndm\_run\_real  
    (ndm\_create\_real\_run\_config), 7  
ndm\_run\_sim  
    (ndm\_create\_real\_run\_config), 7  
ndm\_set\_runtime\_globals  
    (ndm\_new\_runtime\_env), 12  
ndm\_tf\_batch\_to\_jax  
    (ndm\_tf\_batch\_to\_r), 16  
ndm\_tf\_batch\_to\_r, 16  
ndm\_tfrecord\_dtype\_map  
    (ndm\_tfrecord\_fields\_real), 16  
ndm\_tfrecord\_fields\_real, 16  
ndm\_tfrecord\_fields\_sim  
    (ndm\_tfrecord\_fields\_real), 16  
ndm\_train, 2  
ndm\_train (ndm\_build\_model), 3  
  
print.ndm\_config (ndm\_create\_config), 6  
print.ndm\_model (ndm\_build\_model), 3  
print.ndm\_model\_spec  
    (ndm\_model\_spec\_presets), 11  
print.ndm\_trained\_model  
    (ndm\_build\_model), 3